AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: Obligatorisk uppgift nr 3 Fil: darwin_rengifo_C_uppgift_3.pdf Version : V1 Datum: 2017-10-30	

Obligatorisk uppgift nr 3

HARDWARE ABSTRACTION LAYER

Sammanfattning:

Rapporten beskriver hur vi kan använda den PIO Registers på en C program och hur hanterar PIO Register värden av ingång och utgång på minne.

Författare: Darwin Rengifo E-postadress :	Uppgift: Obligatorisk uppgift nr 3 Fil: darwin_rengifo_C_uppgift_3.pdf Version : V1 Datum: 2017-10-30

INNEHÅLL

1. Kravspecifikation.....3

2. Sammanfattning av teoriavsnitt 3.....3

3. Genomföring av CASE 1C.....4

4. Exekverande debuggern.....5

4.1 När knappen KEY_0 är nedtryckt.....5

4.2 När knappen KEY_1 är nedtryckt.....6

5. Används #include "altera_avalon_pio_regs.h".....7

Författare: Darwin Rengifo E-postadress :	Uppgift: Obligatorisk uppgift nr 3 Fil: darwin_rengifo_C_uppgift_3.pdf Version : V1 Datum: 2017-10-30

1. Kravspecifikation

Tabell 1 Kravspecifikation från kund

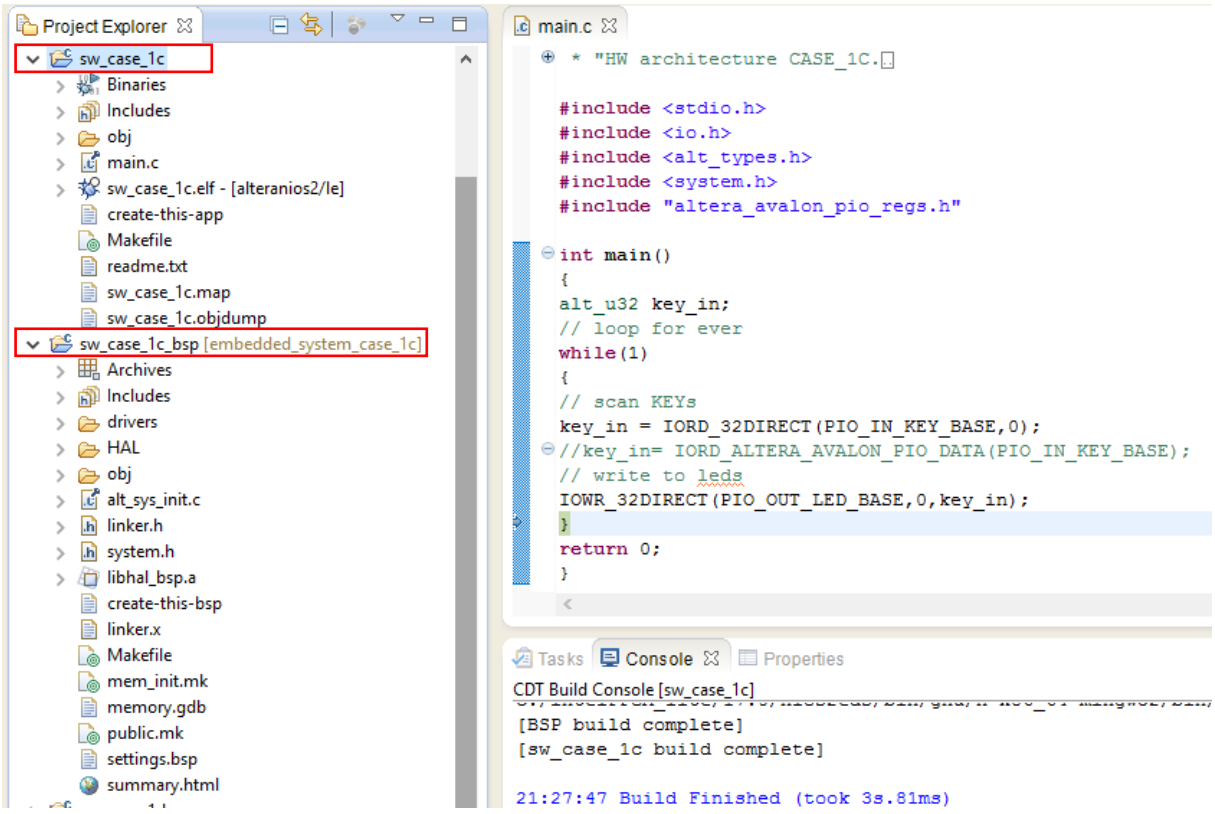
Krav	Beskrivning	Utfört, (ja/nej)
	Förstudie	
Krav_001	Läs igenom teorin som hänger ihop med CASE 1C och genomför CASE 1C.	Ja = Se kapitel 3
	Verktygshantering	
Krav_002	Stega igenom programmet i bilagan med debuggern och visa resultatet med skärmdumpar från PIO dataregister (minnesadresserna) och variabeln ”key_in”. Minst två varv med olika knappnertryckningar.	Ja = Se kapitel 4
Krav_003	Redovisa hur koden ändras om header-filen **#include** "altera_avalon_pio_regs.h" används. Redovisa med en kort förklaring i ett separat kapitel.	Ja = Se kapitel 5
	Leverans	
Krav_004	Leveransen ska ske till plattformen Itslearning. Leveransen ska vara en kort rapport. Namnet på filen ska vara ”fornamn_efternamn_C_uppgift_3”. Sista leveransdag se kursschemat (för VG).	Ja

2. Sammanfattning av teoriavsnitt 3

Det här teoriavsnittet beskriver funktionen av HAL(Hardware Abstraction Layer) för att jobba som gränssnitt mellan software och hardware. Det visar de symboliska definitionerna som finns i huvudfilen system.h, denna fil innehåller listan av perifera installerats på NIOS II Processor.
Vi kan se hur kan används PIO dataregister på en C program och sin beteende genom debuggern när man läser och skriver värdet på sitt adressminne.

3. Genomföring av CASE 1C

- Jag har implementerat CASE 1C som ”sw_case_1c” och filen ”sw_case_1c_bsp”(BSP= Board Support Package).
- Jag har adderat koden som visar teorin som hänger ihop med CASE 1C.
- Till slut ”Build Project”. (Se figur 1.)



Figur 1: Visar genomföring av CASE 1C

4. Exekverande debuggern

När jag köra programmet med debuggern utan trycka knapparna variabeln key_in och PIO dataregister har samma värde som är 3, men när jag trycka knapparna värdena ändras som visar bilderna.

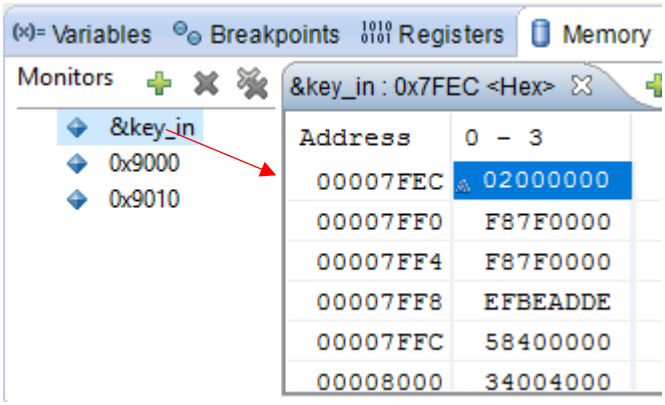
4.1 KEY_0: Lysdioder-LEDR1

1. Variabeln **key_in** har som minnesadress **”0007FEC”** och skriver på den adress värdet 0x0200 = 2; när jag tryckknapp KEY_0 på kortet DE10-Lite. (Se figur 2.)

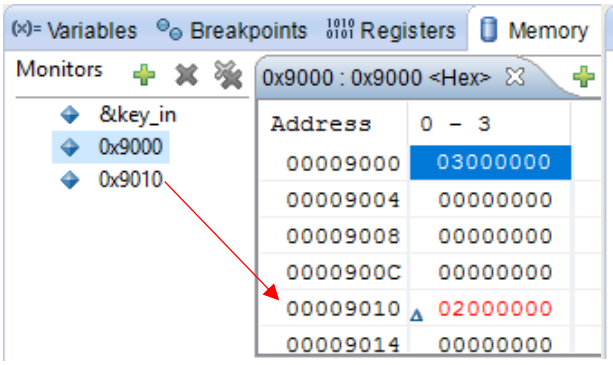
2. **key_in = IORD_32DIRECT(PIO_IN_KEY_BASE,0) ➔0x9010** : key_in läser från PIO värdet 0x0200 = 2. (Se figur 3.)

3. **IOWR_32DIRECT(PIO_OUT_LED_BASE,0,key_in)➔0x9000 :** Det skriver värdet 0x0200 = 2 på key_in. (Se figur 4.)

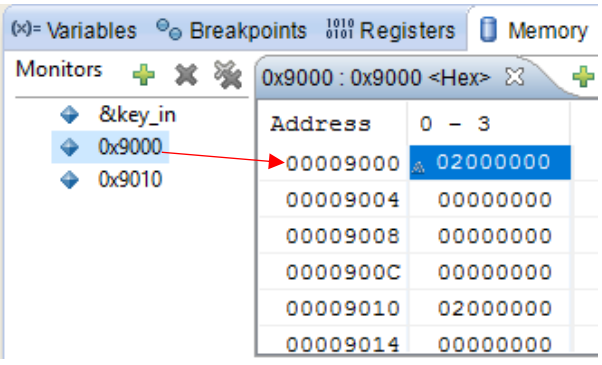
Om jag köra programmen två eller mer gånger laser och skriver samma värde på PIOs registers.



Figur 2: Visar hur PIOs hantera värdet när knappen KEY_0 är nedtryckt.



Figur 3: PIO_IN_KEY_BASE 0x9010



Figur 4: PIO_OUT_LED_BASE 0x9000

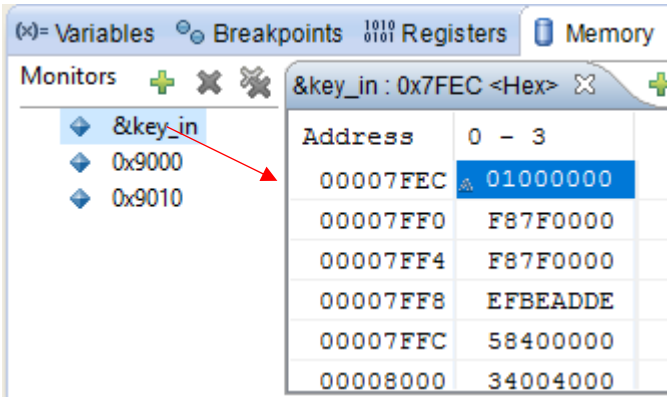
4.2 KEY_1: Lysdioder-LEDR0

1. Variabeln **key_in** har som minnesadress **"0007FEC"** och skriver på den adress värdet 0x0100 = 1; när jag tryckknapp KEY_0 på kortet DE10-Lite. (Se figur 5.)

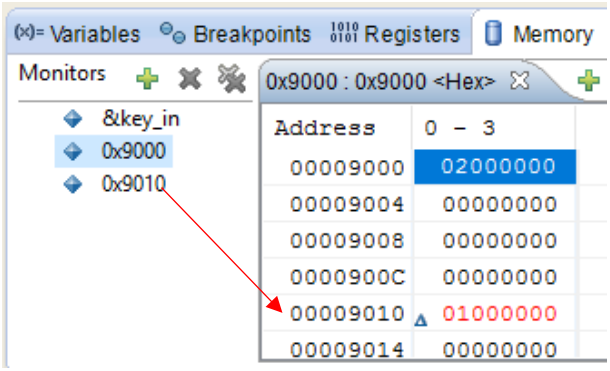
2. **key_in = IORD_32DIRECT(PIO_IN_KEY_BASE,0) ➔0x9010** : key_in läser från PIO värdet 0x0100 = 1. (Se figure 6)

3. **IOWR_32DIRECT(PIO_OUT_LED_BASE,0,key_in)➔0x9000 :** Det skriver värdet 0x0100 = 1 på key_in. (Se figure 7.)

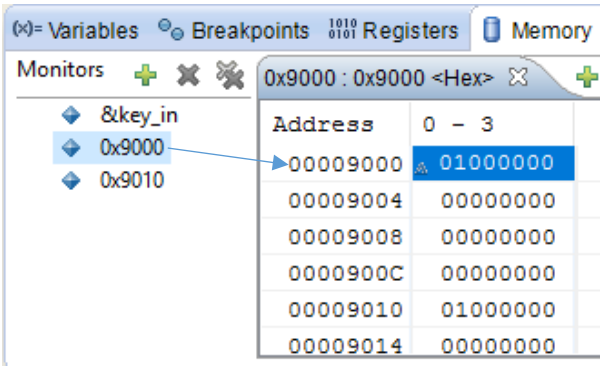
Om jag köra programmen två eller mer gånger läser och skriver samma värde på PIOs registers.



Figur 5: Visar hur PIOs hantera värdet när knappen KEY_1 är nedtryckt.



Figur 6: PIO_IN_KEY_BASE 0x9010



Figur 7: PIO_OUT_LED_BASE 0x9000

5. Används #include ”altera_avlon_pio_regs.h”

När jag inkluderar filen "#include altera_avalon_pio_regs.h" på header och körde jag debuggern det blev samma resultat som visar figurer i kapitel 4.1 och 4.2